In [10]:
```
import tensorflow as tf
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense, Dropout
```

In [11]:
```
VOCAB_SIZE = 10000
MAX_LENGTH = 200
EMBEDDING_DIM = 32
RNN_UNITS = 32
```

In [12]:
```
(x_train, y_train), (x_test, y_test) = imdb.load_data(
    num_words=VOCAB_SIZE
)

print("Training samples:", len(x_train))
print("Testing samples :", len(x_test))
```

```
Training samples: 25000
Testing samples : 25000
```

In [13]:
```
x_train = pad_sequences(
    x_train,
    maxlen=MAX_LENGTH,
    padding="post",
    truncating="post"
)

x_test = pad_sequences(
    x_test,
    maxlen=MAX_LENGTH,
    padding="post",
    truncating="post"
)

print("Training shape:", x_train.shape)
print("Testing shape :", x_test.shape)
```

```
Training shape: (25000, 200)
Testing shape : (25000, 200)
```

In [26]:
```
model = Sequential([
    Embedding(
        input_dim=VOCAB_SIZE,
        output_dim=EMBEDDING_DIM,
        input_length=MAX_LENGTH
    ),

    SimpleRNN(RNN_UNITS),

    Dropout(0.2),

    Dense(1, activation="sigmoid")
])
```

```
C:\Users\cs2\AppData\Roaming\Python\Python313\site-packages\keras\src\layers\core
\embedding.py:123: UserWarning: Argument `input_length` is deprecated. Just remov
e it.
  warnings.warn(
```

In [15]:
```
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)


model.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape
embedding_3 (Embedding)	?
simple_rnn_1 (SimpleRNN)	?
dropout_3 (Dropout)	?
dense_3 (Dense)	?

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

In [16]:
```
history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.2
)
```

```
Epoch 1/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - accuracy: 0.4981 - loss: 0.6960 - val
_accuracy: 0.5126 - val_loss: 0.6924
Epoch 2/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - accuracy: 0.5829 - loss: 0.6747 - val
_accuracy: 0.5130 - val_loss: 0.6973
Epoch 3/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 4s 23ms/step - accuracy: 0.6422 - loss: 0.6234 - val
_accuracy: 0.4924 - val_loss: 0.7167
Epoch 4/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - accuracy: 0.7151 - loss: 0.5101 - val
_accuracy: 0.4988 - val_loss: 0.8025
Epoch 5/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - accuracy: 0.7724 - loss: 0.4159 - val
_accuracy: 0.5034 - val_loss: 0.8739
```

In [17]:
```
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
```

```
)

print("\nTest Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

```
Test Loss: 0.8719062209129333
Test Accuracy: 0.4986799955368042
```

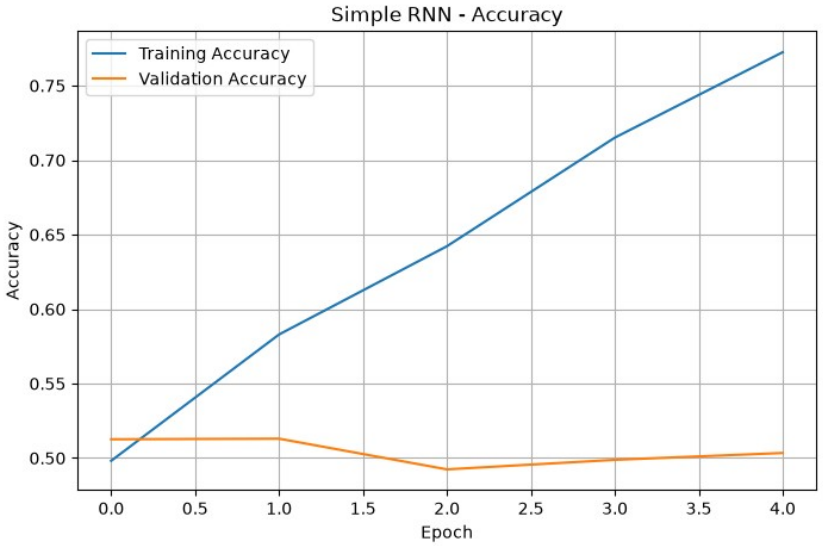
In [18]:
```
plt.figure(figsize=(8, 5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title("Simple RNN - Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.show()
```



In [19]:
```
plt.figure(figsize=(8, 5))

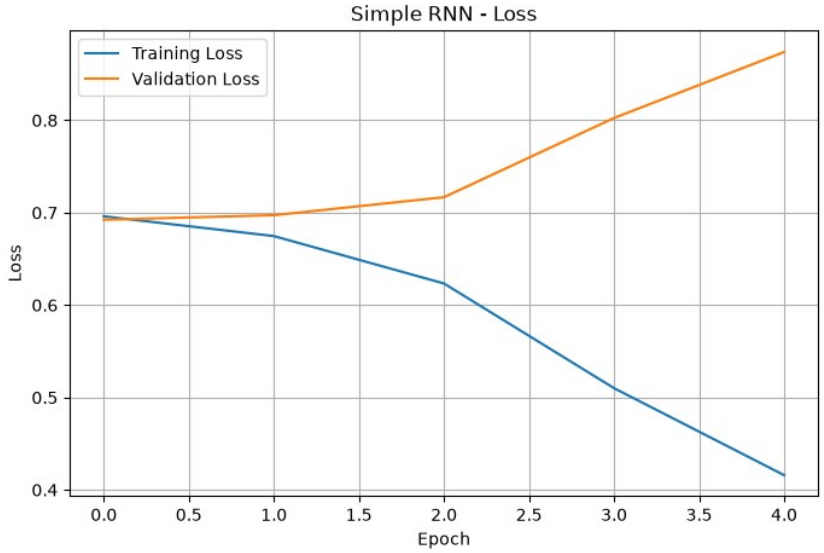
plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
```

```
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title("Simple RNN - Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.show()
```



In [22]:
```
word_index = imdb.get_word_index()

word_index = {
    word: index + 3
    for word, index in word_index.items()
}

word_index["<PAD>"] = 0
word_index["<START>"] = 1
word_index["<UNK>"] = 2
word_index["<UNUSED>"] = 3


def encode_review(review):
    words = review.lower().split()

    encoded_review = []

    for word in words:
        word = word.strip(".,!?;:\"'()[]{}")

        if word in word_index:
            encoded_review.append(word_index[word])
        else:
```

```
            encoded_review.append(2)

    encoded_review = pad_sequences(
        [encoded_review],
        maxlen=MAX_LENGTH,
        padding="post",
        truncating="post"
    )

    return encoded_review


def predict_review(review):
    encoded_review = encode_review(review)

    probability = model.predict(
        encoded_review,
        verbose=0
    )[0][0]

    if probability >= 0.5:
        sentiment = "Positive"
    else:
        sentiment = "Negative"

    print("\nReview:", review)
    print("Prediction:", sentiment)
    print("Positive probability:", round(float(probability), 4))
    print("Negative probability:", round(float(1 - probability), 4))


review = input("Enter a movie review: ")

predict_review(review)
```

```
Review: This movie was boring and terrible. I did not enjoy it
Prediction: Negative
Positive probability: 0.4491
Negative probability: 0.5509
```

In [27]:
```
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

y_pred = model.predict(x_test)

y_pred = (y_pred >= 0.5).astype(int).flatten()

print("Classification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=["Negative", "Positive"]
    )
)

cm = confusion_matrix(y_test, y_pred)

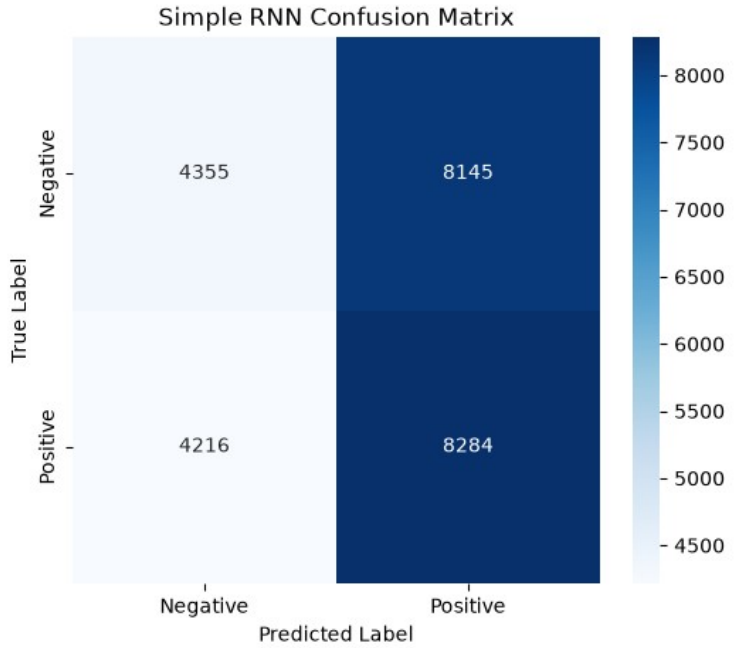
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Negative", "Positive"],
    yticklabels=["Negative", "Positive"]
)

plt.title("Simple RNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step
Classification Report:
              precision    recall  f1-score   support

    Negative       0.51      0.35      0.41     12500
    Positive       0.50      0.66      0.57     12500

    accuracy                           0.51     25000
   macro avg       0.51      0.51      0.49     25000
weighted avg       0.51      0.51      0.49     25000
```



In [ ]:
```
.
```